

Exercices sur les arbres

Exercice 5.1

Arbres n-aires (OCaml)

On s'intéresse dans cet exercice aux *arbres n-aires*, c'est-à-dire aux arbres enracinés dans lesquels chaque nœud peut avoir un nombre quelconque de fils.

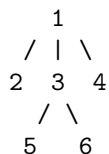
On adopte la représentation OCaml suivante :

```
type arbre =
  | Vide
  | Noeud of int * arbre list
```

Dans le constructeur `Noeud(v,l)`, `v` désigne l'étiquette de la racine et `l` la liste de ses sous-arbres.

Exemple.

On considère l'arbre `t` suivant :



Il est représenté en OCaml par :

```
let t =
  Noeud (1, [
    Noeud (2, []);
    Noeud (3, [
      Noeud (5, []);
      Noeud (6, [])
    ]);
    Noeud (4, [])
  ])
])
```

Dans la suite, on pourra vérifier les fonctions demandées sur cet arbre.

1) Écrire une fonction `taille : arbre -> int` qui renvoie le nombre total de nœuds de l'arbre passé en argument.

Par exemple, pour l'arbre `t`, on doit obtenir : `taille t = 6`

2) Écrire une fonction `hauteur : arbre -> int` qui renvoie la hauteur de l'arbre. On précisera la valeur renvoyée pour un arbre vide.

Pour l'arbre `t`, la hauteur vaut : `hauteur t = 2`

3) Écrire une fonction `contient : arbre -> int -> bool` de sorte que `contient a x` renvoie `true` si une étiquette `x` donnée apparaît dans l'arbre `a`.

On vérifiera notamment que : `contient t 5 = true`, `contient t 7 = false`

4) Écrire une fonction `nb_occurrences : arbre -> int -> int` de sorte que `nb_occurrences a x` renvoie le nombre d'occurrences de l'étiquette `x` dans l'arbre `a`.

Par exemple :

`nb_occurrences t 3 = 1`, `nb_occurrences t 8 = 0`.

5) On souhaite maintenant déterminer le chemin menant à une étiquette donnée.

a) Écrire une fonction `chemin : arbre -> int -> int list option` telle que `chemin a x` renvoie une option donnant un chemin depuis la racine vers l'étiquette `x` dans l'arbre `a`, s'il en existe et `None` sinon.

On devra obtenir :

`chemin t 6 = Some [1;3;6]`, `chemin t 9 = None`

L'étiquette recherchée peut apparaître plusieurs fois dans l'arbre.

b) Écrire une fonction `chemins : arbre -> int -> int list list` qui renvoie la liste de tous les chemins possibles.

Dans l'arbre `t`, on aura par exemple : `chemins t 2 = [[1;2]]`

6) Écrire une fonction `feuilles : arbre -> int list` qui renvoie une liste contenant les étiquettes de toutes les feuilles possibles. Il peut y avoir des doublons dans cette liste.

Pour l'arbre `t` : `feuilles t = [2;5;6;4]`

- 7) a) Écrire une fonction `parcours_prefixe : arbre -> int list` qui renvoie la liste des noeuds dans le parcours préfixe.
On devra obtenir : `parcours_prefixe t = [1;2;3;5;6;4]`
- b) Écrire une fonction `parcours_postfixe : arbre -> int list` qui renvoie la liste des noeuds dans le parcours postfixe.
On devra obtenir : `parcours_postfixe t = [2;5;6;3;4;1]`
- 8) Écrire une fonction `etiquettes_niveau : arbre -> int -> int list` qui renvoie toutes les étiquettes d'un niveau donné.

Par exemple :

`etiquettes_niveau t 1 = [2;3;4],      etiquettes_niveau t 2 = [5;6]`

- 9) On dit qu'un arbre est *uniforme* si tous ses nœuds internes ont le même nombre de fils.
Écrire une fonction `est_uniforme : arbre -> bool`.
L'arbre t n'est pas uniforme.
- 10) Écrire une fonction `degre_max : arbre -> int` qui renvoie le nombre maximal de fils d'un nœud de l'arbre passé en argument.
Pour l'arbre t : `degre_max t = 3`
- 11) Écrire une fonction `sous_arbre : arbre -> int -> arbre option` qui renvoie le sous-arbre enraciné au premier nœud portant l'étiquette donnée.
Par exemple, `sous_arbre t 3` renvoie l'arbre de racine 3 contenant les nœuds 5 et 6.
- 12) Écrire une fonction `ppac : arbre -> int -> int -> int option` qui renvoie le *plus petit ancêtre commun* de deux étiquettes.
On devra obtenir : `ppac t 5 6 = Some 3,      ppac t 2 6 = Some 1.`

Exercice 5.2

Autour des tas binomiaux (OCaml)

Dans cet exercice, on s'intéresse à l'étude et à l'implémentation des *tas binomiaux*.

Sauf mention explicite, on n'utilisera **ni références mutables, ni tableaux, ni boucles impératives**. Les fonctions demandées devront être récursives et utiliser des structures de données immuables.

I Arbres binomiaux

Définition 1 (Arbre enraciné). Un *arbre enraciné* est un graphe orienté acyclique possédant une unique racine et tel que tout nœud, sauf la racine, possède un unique parent.

On choisit la représentation OCaml suivante pour coder des arbres enracinés à valeurs entières :

```
type arbre =
  | Vide
  | Noeud of int * arbre list
```

Dans le constructeur `Noeud(v, l)`, l'entier v désigne la valeur de la racine et l la liste de ses fils.

- 1) Écrire les fonctions OCaml suivantes :

- `vide : arbre -> bool`, qui renvoie `true` si l'arbre est vide ;
- `racine : arbre -> int`, qui renvoie la valeur de la racine d'un arbre non vide ;
- `fils : arbre -> arbre list`, qui renvoie la liste des fils de la racine.

On précisera le comportement de ces fonctions lorsque l'arbre est vide.

Définition 2 (Arbre binomial). Un *arbre binomial d'ordre* $k \geq 0$ est un arbre enraciné dont les fils sont ordonnés, défini récursivement par :

- un arbre binomial d'ordre 0 est constitué d'un unique nœud sans fils ;
- pour $k \geq 1$, un arbre $a_k = (r, [t_{k-1}, \dots, t_0])$ est un arbre binomial d'ordre k si :
 - t_{k-1} est un arbre binomial d'ordre $k-1$;
 - $(r, [t_{k-2}, \dots, t_0])$ est un arbre binomial d'ordre $k-1$;
 - la racine de t_{k-1} a une valeur supérieure ou égale à r .

- 2) Écrire une fonction OCaml `fusion_arbre : arbre -> arbre -> arbre` qui construit, à partir de deux arbres binomiaux d'ordre $k-1$, un arbre binomial d'ordre k contenant exactement les mêmes valeurs. On notera cette opération $a_1 \oplus a_2$.

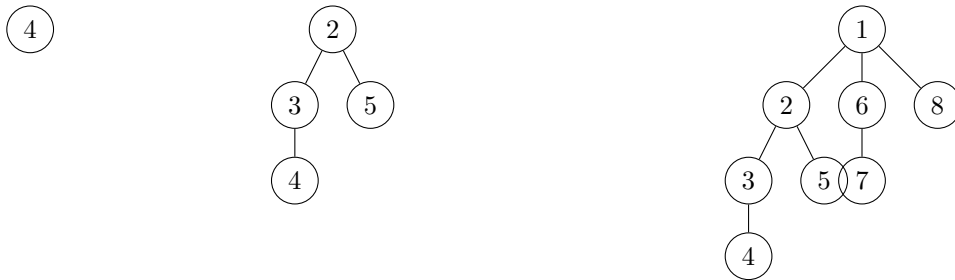
- 3) a) Montrer par récurrence que la racine d'un arbre binomial d'ordre k possède exactement k fils.
 b) Écrire une fonction OCaml `ordre : arbre -> int` qui renvoie l'ordre d'un arbre binomial non vide.
- 4) Montrer qu'un arbre binomial d'ordre k possède exactement 2^k nœuds.
- 5) Écrire une fonction récursive `est_arbre_binomial : arbre -> bool` qui renvoie `true` si l'arbre donné est un arbre binomial, `false` sinon.

II Tas binomiaux

Définition 4 (Tas binomial). Un *tas binomial* est une collection d'arbres binomiaux, au plus un par ordre. On représente un tas binomial par une liste d'arbres, où l'arbre en position i est soit vide, soit un arbre binomial d'ordre i :

```
type tas = arbre list
```

Par exemple, les trois arbres qui suivent forment un tas binomial.



Définition 8 (Signature d'un tas). Soit $T = [a_0; \dots; a_k]$ un tas binomial. On appelle *signature* de T la suite $(s_0, \dots, s_k)$ définie par :

$$s_i = \begin{cases} 0 & \text{si } a_i \text{ est vide,} \\ 1 & \text{sinon.} \end{cases}$$

- 6) Exprimer le nombre total de nœuds d'un tas T à partir de sa signature. Montrer que, si T est de longueur $k+1$,

$$2^k \leq |T| < 2^{k+1},$$

et en déduire k en fonction de $|T|$. Comment retrouver la signature du tas en connaissant le nombre d'éléments dans le tas?

- 7) Écrire une fonction OCaml `minimum_tas : tas -> int` qui renvoie la valeur minimale contenue dans un tas non vide, et donner sa complexité en fonction de $|T|$.

III Insertion dans un tas binomial

Insertion dans un tas binomial. L'insertion d'un élément p dans un tas se fait en créant un arbre binomial d'ordre 0, puis en fusionnant tant qu'il existe une collision d'ordres.

- 8) Écrire une fonction OCaml `insertion : int -> tas -> tas` telle que `insertion p t` insère l'élément p dans le tas t .
- 9) Donner la complexité de l'insertion en fonction du nombre total de nœuds du tas.
- 10) Exprimer la signature du tas obtenu après insertion de p en fonction de la signature initiale de T .

IV Suppression d'un élément dans un tas binomial

Suppression du minimum. La suppression d'un élément prioritaire (c'est-à-dire minimal) dans un tas binomial consiste à supprimer l'arbre dont la racine est minimale, puis à réinsérer ses sous-arbres dans le tas restant.

- 11) Décrire le principe algorithmique de la suppression du minimum dans un tas binomial.
- 12) Écrire une fonction OCaml `extraire_minimum : tas -> int * tas` qui renvoie un couple constitué :
- de la valeur minimale contenue dans le tas ;
 - du tas binomial obtenu après suppression de cet élément.
- 13) Donner la complexité de la suppression du minimum en fonction de $|T|$.